

FORMATION EMBARQUÉE

TP4 — Seance 3

Formation Systèmes embarqués & programmation bas niveau

Systèmes embarqués · bas niveau · automatismes

TP 4 — Fiche de séance 3 : client de supervision PC (3 h)

En-tête pédagogique

Objectifs	Lire/écrire un automate en Modbus TCP depuis un PC ; surveiller un mot de vie ; journaliser ; faire des essais croisés automate ↔ PC
Prérequis	Séance 2 : table Modbus implémentée et testée
Outils	Machine Expert Basic + simulateur ; Python + <code>pip install pymodbus</code>
Livrable	<code>supervision.py</code> fonctionnel + 5 essais croisés documentés

Déroulé minuté

0:00-0:20 — Vérifier le lien avec un client générique d'abord

Avant d'écrire du code, valide le bus avec un outil tout fait (méthode : une inconnue à la fois). QModMaster ou `mbpoll` :

```
mbpoll -m tcp -a 1 -r 100 -c 5 -t 4 192.168.1.50 # lit %MW100..104
```

Tu dois voir les 5 mots. Si le simulateur n'expose pas le port 502, active le serveur Modbus dans sa config. **Ce test isole** : si `mbpoll` marche mais pas ton script, le bug est dans ton script ; sinon, dans l'automate.

0:20-1:30 — Écrire `supervision.py`

Pars du corrigé complet fourni : `code/python/supervision.py` . Il fait déjà tout (lecture, affichage tableau de bord, mot de vie, CSV, commandes clavier en thread). Ton travail : le **comprendre ligne par ligne** et l'adapter à ta table. Points à saisir :

- La lecture groupée `read_holding_registers(address=100, count=5)` : un seul échange réseau pour 5 mots cohérents (mieux que 5 échanges).
- Le **décodage des bits** d'état avec des masques (`etat & 0x01` pour P1) — exactement le C du module 01, en Python.
- La **surveillance du mot de vie** : compteur figé 3 fois → alarme.
- Le thread clavier : les commandes ne bloquent jamais la scrutation.
- L'écriture `write_register(111, valeur)` : le PC envoie une **demande**, l'automate borne et valide (jamais l'inverse).

Lance-le : `python3 supervision.py 192.168.1.50` .

1:30-2:30 — Les 5 essais croisés (le cœur du TP)

Déroule et **documente chacun** (capture + observation) :

#	Scénario	Manip	Attendu
1	Cycle normal	laisser tourner	niveau oscille 40 ↔ 60, heures P1/P2 s'équilibrent
2	Consigne	taper <code>consigne 650</code>	seuil haut passe à 65 % ; <code>consigne 950</code> → borné à 80 %
3	Panne CPU	mettre le simulateur en STOP	le PC détecte le mot de vie figé < 3 s, alarme affichée
4	Commande stop	taper <code>stop</code> pendant un remplissage	pompes coupées PAR L'AUTOMATE (le PC n'a écrit qu'une demande)
5	Capteur figé	bloquer <code>%IW0.0</code> pendant qu'une pompe tourne	DEF_CAPTEUR après 30 s, visible côté PC, positions sûres

L'essai 3 est le plus formateur : c'est lui qui distingue une vraie supervision d'une démo. Un lien TCP « connecté » ne prouve pas que l'automate vit. Le mot de vie est la seule garantie.

✓ **Point de contrôle** : les 5 essais documentés, essai 3 en évidence.

2:30-2:55 — (option) Version Java

Remplace le client Python par ton programme Java du module 05 §7.1 (avec la bibliothèque j2mod) : **même table, même comportement**. Constat à écrire : la table d'échange documentée rend le langage du client indifférent — c'est exactement son rôle. Python pour prototyper, Java pour un superviseur d'entreprise, le contrat Modbus ne change pas.

2:55-3:00 — Barème final

Remplis la grille /20 du TP principal ([tp4-schneider-cuve.md](https://github.com/tp4-schneider-cuve)) avec preuves. Commit du script + journal CSV d'exemple + document de table.

Erreurs fréquentes

Symptôme	Cause	Remède
<code>Connection refused</code>	serveur Modbus non activé sur le simulateur	activer dans la config M221
pymodbus : <code>.registers</code> vide	<code>.isError()</code> non testé, adresse hors zone	tester l'erreur, vérifier l'adresse
Le PC croit tout OK alors que la CPU est STOP	mot de vie non surveillé	comparer à la valeur précédente
<code>stop</code> ne coupe pas les pompes	le PC écrit une %Q au lieu d'une demande	le PC écrit %MW110, l'API décide
Valeurs ×10 mal affichées	oubli de diviser par 10 à l'affichage	<code>niveau/10</code>

Bilan du TP 4

Tu as construit une chaîne **automate** ↔ **supervision** complète et vu les deux écosystèmes (TIA au TP 3, EcoStruxure ici) sur la même méthode. Le profil « automaticien qui parle aussi informatique » (Modbus, Python/Java, tables d'échange) est très recherché. Range ce projet dans ton portfolio à côté du TP 3 : ensemble, ils montrent que tu maîtrises la démarche, pas juste un logiciel.

 Retour : [TP 4 \(vue d'ensemble\)](#) · [Module 10 — STM32](#)